SVKM's NMIMS
Mukesh Patel School of Technology Management & Engineering
Computer Engineering Department
Program: MCA, Semester - I

Course: Java Programming

Experiment No.02

PART A

(PART A : TO BE REFFERED BY STUDENTS)

A.1 Aim: To understand and implement concepts like control statements and arrays in Java.

TASK 1:

Problem Description: Consider the scenario of processing marks of a student John for a course as part of a student management system. The assumption is that John takes exams in five different subjects. Find out the grade scored by John as per the table given below.

Module Test Marks	Grade
>= 80	A
>=73 and < 80	B+
>=65 and < 73	B
>=55 and < 65	C

Task 2: Write a Java program print "Hello" for *n* times.

Task 3: Write a Java program that prints multiplication table of a number given by the user.

Task 4: Write a Java program to find factorial of a number entered by the user.

Task 5: Write a Java program to print the sum of series as below. The user will give value of n.

Sum1= 1 + 1/2 + 1/3 + …… + 1/n

Sum2 = $1+ 2^2 + 3^2 + 4^2 + \ldots\ldots n^n$

Task 6:

Problem Description: The finance department of a company wants to calculate the monthly net pay of one of its employee by finding the income tax to be paid (in Indian Rupees) and the net salary after the income tax deduction. The employee should pay income tax if his monthly gross salary is more than Rs. 10,000 (Indian Rupees) and the percentage of income tax should be considered as 20% of the gross salary. Display the employee id, basic salary, allowances, gross pay, income tax and net pay.

Gross Salary (In Indian Rupees)	Income Tax percentage
Below 5,000	Nil
5,001 to 10,000	10 %
10,001 to 20,000	20%
More than 20,000	30%

Task 7: Write a Java program to store marks of 10 students and display them.

Task 8: Write a Java program to find sum and average of marks stored in an array in task 7.

Task 9: Write a Java program to store marks of five students for three subjects. Find maximum marks scored by each student, display the Name, Subject and Marks

Task 10: Write a Java program to read two matrices, multiply them and display the result.

A.2 Prerequisite:

1. Fundamental concepts of C\C++.
2. Control statements in Java

A.3 Outcome:

After successful completion of this experiment, students will be able to

1. Implement concept of different types of control statements in Java

A.4 Theory:

A.4.1.

Java Control Statements:

Java compiler executes the code from top to bottom. The statements in the code are executed according to the order in which they appear. However, Java provides statements that can be used to control the flow of Java code. Such statements are called control flow statements. It is one of the fundamental features of Java, which provides a smooth flow of program.

Java provides three types of control flow statements.

1. Decision Making statements
    - if statements
    - switch statement
2. Loop statements
    - do while loop
    - while loop
    - for loop
    - for-each loop
3. Jump statements
    - break statement
    - continue statement

Java Arrays:

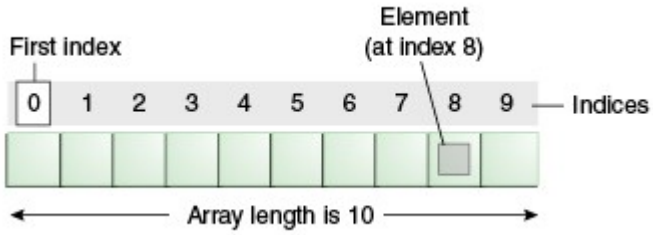
Normally, an array is a collection of similar type of elements which has contiguous memory location.

Java array is an object which contains elements of a similar data type. Additionally, The elements of an array are stored in a contiguous memory location. It is a data structure where we store similar elements. We can store only a fixed set of elements in a Java array.

Array in Java is index-based, the first element of the array is stored at the 0th index, 2nd element is stored on 1st index and so on.

Unlike C/C++, we can get the length of the array using the length member. In C/C++, we need to use the sizeof operator.

In Java, array is an object of a dynamically generated class. Java array inherits the Object class, and implements the Serializable as well as Cloneable interfaces. We can store primitive values or objects in an array in Java. Like C/C++, we can also create single dimentional or multidimentional arrays in Java. Moreover, Java provides the feature of anonymous arrays which is not available in C/C++.



PART B

(PART B : TO BE COMPLETED BY STUDENTS)

Students must submit the soft copy as per following segments within two hours of the practical. The soft copy must be uploaded on the portal at the end of the practical. The filename should be **rollno_subject_experimentno Example: C001_JP_Exp2**

Roll No. A012	Name: Brinda Das
Class : MCA	Batch : B1
Date of Experiment: 03-08-2026	Date of Submission: 03-08-2026
Grade :	

B.1 Software Code written by student along with the output:

Task 1:

Problem Description: Consider the scenario of processing marks of a student John for a course as part of a student management system. The assumption is that John takes exams in five different subjects. Find out the grade scored by John as per the table given below.

Module Test Marks	Grade
>= 80	A
>=73 and < 80	B+
>=65 and < 73	B
>=55 and < 65	C

File name: task1.java

Code:

```
import java.util.Scanner;
public class task1 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        double[] marks = new double[5];
        double total = 0;
        System.out.println("Enter John's marks for 5 subjects:");

        for (int i = 0; i < 5; i++) {
            System.out.print("Subject " + (i + 1) + ": ");
            marks[i] = scanner.nextDouble();
            total += marks[i];
        }
        double average = total / 5;
        System.out.println("\nJohn's Result");

        for (int i = 0; i < 5; i++) {
            String grade = "N/A";

            if (marks[i] >= 80) {
                grade = "A";
            } else if (marks[i] >= 73 && marks[i] < 80) {
                grade = "B+";
            } else if (marks[i] >= 65 && marks[i] < 73) {
                grade = "B";
            } else if (marks[i] >= 55 && marks[i] < 65) {
                grade = "C";
            }
            else if (marks[i] >= 40 && marks[i] < 55) {
                grade = "D";
            }
            else{
                grade = "F";
            }
            System.out.println("Subject " + (i + 1) + " Marks: " + marks[i] + " -
Grade: " + grade);
        }

        System.out.println("Total Marks: " + total);
        System.out.println("Average Marks: " + average);
        scanner.close();
```

```
    }
}
```

Output:

```
D:\Java\Java P2 - Code>javac task1.java

D:\Java\Java P2 - Code>java task1
Enter John's marks for 5 subjects:
Subject 1: 70
Subject 2: 66
Subject 3: 78
Subject 4: 99
Subject 5: 86

John's Result
Subject 1 Marks: 70.0 - Grade: B
Subject 2 Marks: 66.0 - Grade: B
Subject 3 Marks: 78.0 - Grade: B+
Subject 4 Marks: 99.0 - Grade: A
Subject 5 Marks: 86.0 - Grade: A
Total Marks: 399.0
Average Marks: 79.8
```

Task 2: Write a Java program print "Hello" for *n* times.

File name: hello.java

Code:

```
import java.util.Scanner;
public class hello
{
    public static void main(String args[])
    {
        int num;
        Scanner scanner = new Scanner(System.in);
        System.out.println("Enter the number of times 'Hello' to be printed: ");
        num = scanner.nextInt();
        for(int i=0; i<=num; i++)
        {
            System.out.println("\nHello");
        }
    }
}
```

Output:

```
F:\Java\Java P2 - Code>javac hello.java

F:\Java\Java P2 - Code>java hello
Enter the number of times 'Hello' to be printed:
4

Hello

Hello

Hello

Hello

Hello
```

Task 3: Write a Java program that prints multiplication table of a number given by the user.

File name: multiply.java

Code:

```
import java.util.Scanner;

public class multiply
{
    public static void main(String args[])
    {
        int num;
        Scanner scanner = new Scanner(System.in);
        System.out.println("Enter a number for multiplication:");
        num = scanner.nextInt();
        System.out.println("The multiplication of "+num+" is: ");
        for(int i=1; i<=10; i++)
        {
            System.out.println(num +" x "+ i +" = "+ num*i);
        }
    }
}
```

Output:

```
F:\Java\Java P2 - Code>javac multiply.java

F:\Java\Java P2 - Code>java multiply
Enter a number for multiplication:
15
The multiplication of 15 is:
15 x 1 = 15
15 x 2 = 30
15 x 3 = 45
15 x 4 = 60
15 x 5 = 75
15 x 6 = 90
15 x 7 = 105
15 x 8 = 120
15 x 9 = 135
15 x 10 = 150
```

Task 4: Write a Java program to find factorial of a number entered by the user.

File name: factorial.java

Code:

```
import java.util.Scanner;

public class factorial

{
    public static void main(String args[])
    {
        int num;
        long fact = 1;
        Scanner scanner = new Scanner(System.in);
        System.out.println("Enter a number for finding the factorial:");
        num = scanner.nextInt();
        for(int i=1; i<=num; i++)
        {
            fact *= i;
        }
        System.out.println("The factorial of "+num+" is "+fact);
    }
}
```

Output:

```
F:\Java\Java P2 - Code>javac factorial.java

F:\Java\Java P2 - Code>java factorial
Enter a number for finding the factorial:
4
The factorial of 4 is 24
```

Task 5: Write a Java program to print the sum of series as below. The user will give value of n.

Sum1= 1 + 1/2 + 1/3 + …… + 1/n

Sum2 = $1+ 2^2 + 3^2 + 4^2 + \ldots\ldots n^n$

File name: SumSeries.java

Code:

```
import java.util.Scanner;
public class SumSeries {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter the value of n: ");
        int n = scanner.nextInt();

        if (n <= 0) {
            System.out.println("Please enter a positive integer greater than
0.");
        } else {
            double[] terms1 = new double[n];
            double[] terms2 = new double[n];

            double sum1 = 0;
            double sum2 = 0;

            for (int i = 1; i <= n; i++) {

                terms1[i - 1] = 1.0 / i;
                sum1 += terms1[i - 1];

                terms2[i - 1] = Math.pow(i, i);
                sum2 += terms2[i - 1];
            }
            System.out.println("\nAnswer:");
            System.out.printf("Sum1 = %.4f\n", sum1);
            System.out.printf("Sum2 = %.0f\n", sum2);
        }
        scanner.close();
    }
}
```

Output:

```
F:\Java\Java P2 - Code>javac SumSeries.java

F:\Java\Java P2 - Code>java SumSeries
Enter the value of n: 7

Answer:
Sum1 = 2.5929
Sum2 = 873612
```

Task 6:

Problem Description: The finance department of a company wants to calculate the monthly net pay of one of its employee by finding the income tax to be paid (in Indian Rupees) and the net salary after the income tax deduction. The employee should pay income tax if his monthly gross salary is more than Rs. 10,000 (Indian Rupees) and the percentage of income tax should be considered as 20% of the gross salary. Display the employee id, basic salary, allowances, gross pay, income tax and net pay.

Gross Salary (In Indian Rupees)	Income Tax percentage
Below 5,000	Nil
5,001 to 10,000	10 %
10,001 to 20,000	20%
More than 20,000	30%

Filename: GrossSalary.java

Code:

```
import java.util.Scanner;

public class GrossSalary {

    public static void main(String args[]) {

        int id;

        double salary;

        double allowances;

        Scanner scanner = new Scanner(System.in);

        System.out.println("Enter employee ID:");

        id = scanner.nextInt();
```

```
System.out.println("Enter salary: ");
salary = scanner.nextDouble();

System.out.println("Enter Allowances: ");
allowances = scanner.nextDouble();

double grossPay = salary + allowances;
double taxPercentage = 0.0;

if (grossPay <= 5000) {
    taxPercentage = 0.0;
    System.out.println("No need to pay any income tax");
}
else if (grossPay >= 5001 && grossPay <= 10000) {
    taxPercentage = 10.0;
    System.out.println("10% tax bracket applied");
}
else if (grossPay >= 10001 && grossPay <= 20000) {
    taxPercentage = 20.0;
    System.out.println("20% tax bracket applied");
}
else if (grossPay >= 20001) {
    taxPercentage = 30.0;
    System.out.println("30% tax bracket applied");
}
else {
    System.out.println("Invalid input of salary");
```

```
        }

        double incomeTax = grossPay * (taxPercentage / 100.0);

        double netPay = grossPay - incomeTax;

        System.out.println("\nEmployee ID: " + id);

        System.out.println("Basic Salary: Rs. " + salary);

        System.out.println("Allowances: Rs. " + allowances);

        System.out.println("Gross Pay: Rs. " + grossPay);

        System.out.println("Income Tax: Rs. " + incomeTax + " (" + taxPercentage
+ "%)");

        System.out.println("Net Pay: Rs. " + netPay);

        scanner.close();

    }

}
```

Output:

```
D:\Java\Java P2 - Code>javac GrossSalary.java

D:\Java\Java P2 - Code>java GrossSalary
Enter employee ID:
12
Enter salary:
25000
Enter Allowances:
1500
30% tax bracket applied

Employee ID: 12
Basic Salary: Rs. 25000.0
Allowances: Rs. 1500.0
Gross Pay: Rs. 26500.0
Income Tax: Rs. 7950.0 (30.0%)
Net Pay: Rs. 18550.0
```

Task 7: Write a Java program to store marks of 10 students and display them.

File name: Student.java

Code:

```
import java.util.Scanner;

public class Student {
    public static void main(String[] args) {
        double[] marks = new double[10];
        Scanner scanner = new Scanner(System.in);
        System.out.println("Enter Marks for 10 Students");

        for (int i = 0; i < marks.length; i++) {
            System.out.print("Enter marks for Student " + (i + 1) + ": ");
            marks[i] = scanner.nextDouble();
        }
        System.out.println("\nDisplaying Student Marks");

        for (int i = 0; i < marks.length; i++) {
            System.out.println("Student " + (i + 1) + " Marks: " + marks[i]);
        }
        scanner.close();
    }
}
```

Output:

```
D:\Java\Java P2 - Code>javac Student.java

D:\Java\Java P2 - Code>java Student
Enter Marks for 10 Students
Enter marks for Student 1: 60
Enter marks for Student 2: 90
Enter marks for Student 3: 87
Enter marks for Student 4: 78
Enter marks for Student 5: 55
Enter marks for Student 6: 49
Enter marks for Student 7: 90
Enter marks for Student 8: 98
Enter marks for Student 9: 71
Enter marks for Student 10: 82

Displaying Student Marks
Student 1 Marks: 60.0
Student 2 Marks: 90.0
Student 3 Marks: 87.0
Student 4 Marks: 78.0
Student 5 Marks: 55.0
Student 6 Marks: 49.0
Student 7 Marks: 90.0
Student 8 Marks: 98.0
Student 9 Marks: 71.0
Student 10 Marks: 82.0
```

Task 8: Write a Java program to find sum and average of marks stored in an array in task 7.

File name: StudentSumAvg.java

Code:

```
import java.util.Scanner;

public class StudentSumAvg{
    public static void main(String[] args) {
        double[] marks = new double[10];
        Scanner scanner = new Scanner(System.in);

        System.out.println("Enter Marks for 10 Students");

        for (int i = 0; i < marks.length; i++) {
            System.out.print("Enter marks for Student " + (i + 1) + ": ");
            marks[i] = scanner.nextDouble();
        }

        double sum = 0;

        for (int i = 0; i < marks.length; i++) {
            sum += marks[i];
        }
        double average = sum / marks.length;
        System.out.println("Total Marks: " + sum);
        System.out.printf("Average Marks: %.2f\n", average);
        scanner.close();
    }
}
```

Output:

```
D:\Java\Java P2 - Code>javac StudentSumAvg.java

D:\Java\Java P2 - Code>java StudentSumAvg
Enter Marks for 10 Students
Enter marks for Student 1: 88
Enter marks for Student 2: 65
Enter marks for Student 3: 67
Enter marks for Student 4: 88
Enter marks for Student 5: 89
Enter marks for Student 6: 64
Enter marks for Student 7: 63
Enter marks for Student 8: 93
Enter marks for Student 9: 91
Enter marks for Student 10: 51
Total Marks: 759.0
Average Marks: 75.90
```

Task 9: Write a Java program to store marks of five students for three subjects. Find maximum marks scored by each student, display the Name, Subject and Marks

File name: MaxStudentMarks.java

Code:

```
import java.util.Scanner;

public class MaxStudentMarks {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        String[] students = {"Student 1", "Student 2", "Student 3", "Student 4",
"Student 5"};
        String[] subjects = {"Subject 1", "Subject 2", "Subject 3"};

        double[][] marks = new double[5][3];

        for (int i = 0; i < 5; i++) {
            System.out.println("Enter marks for " + students[i]);
            for (int j = 0; j < 3; j++) {
                System.out.print("Enter marks for " + subjects[j] + ": ");
                marks[i][j] = scanner.nextDouble();
            }
            System.out.println();
        }

        System.out.println("The maximum marks for each student in each subject");
        for (int i = 0; i < 5; i++) {
            double maxMarks = marks[i][0];
            int maxSubjectIndex = 0;

            for (int j = 1; j < 3; j++) {
                if (marks[i][j] > maxMarks) {
                    maxMarks = marks[i][j];
                    maxSubjectIndex = j;
                }
            }
                 System.out.println(students[i] + " has the highest in " +
subjects[maxSubjectIndex] + ": " + maxMarks);
        }

        scanner.close();
    }
}
```

Output:

```
D:\Java\Java P2 - Code>javac MaxStudentMarks.java

D:\Java\Java P2 - Code>java MaxStudentMarks
Enter marks for Student 1
Enter marks for Subject 1: 60
Enter marks for Subject 2: 54
Enter marks for Subject 3: 66

Enter marks for Student 2
Enter marks for Subject 1: 60
Enter marks for Subject 2: 70
Enter marks for Subject 3: 43

Enter marks for Student 3
Enter marks for Subject 1: 65
Enter marks for Subject 2: 67
Enter marks for Subject 3: 77

Enter marks for Student 4
Enter marks for Subject 1: 87
Enter marks for Subject 2: 54
Enter marks for Subject 3: 45

Enter marks for Student 5
Enter marks for Subject 1: 76
Enter marks for Subject 2: 75
Enter marks for Subject 3: 77

The maximum marks for each student in each subject
Student 1 has the highest in Subject 3: 66.0
Student 2 has the highest in Subject 2: 70.0
Student 3 has the highest in Subject 3: 77.0
Student 4 has the highest in Subject 1: 87.0
Student 5 has the highest in Subject 3: 77.0
```

Task 10: Write a Java program to read two matrices, multiply them and display the result.

File name: matrix.java

Code:

```
import java.util.Scanner;

public class matrix {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);



        System.out.print("Enter rows for Matrix A: ");

        int r1 = scanner.nextInt();

        System.out.print("Enter columns for Matrix A: ");
```

```
        int c1 = scanner.nextInt();

        System.out.print("Enter rows for Matrix B: ");
        int r2 = scanner.nextInt();
        System.out.print("Enter columns for Matrix B: ");
        int c2 = scanner.nextInt();

        if (c1 != r2) {
            System.out.println("\nMatrix multiplication is NOT possible!");
            System.out.println("Columns of Matrix A must equal Rows of Matrix
B.");
            scanner.close();
            return;
        }

        int[][] A = new int[r1][c1];
        int[][] B = new int[r2][c2];
        int[][] C = new int[r1][c2];

        System.out.println("\nEnter elements for Matrix A");
        for (int i = 0; i < r1; i++) {
            for (int j = 0; j < c1; j++) {
                System.out.print("A[" + i + "][" + j + "]: ");
                A[i][j] = scanner.nextInt();
            }
        }
```

```
        System.out.println("\nEnter elements for Matrix B");
        for (int i = 0; i < r2; i++) {
            for (int j = 0; j < c2; j++) {
                System.out.print("B[" + i + "][" + j + "]: ");
                B[i][j] = scanner.nextInt();
            }
        }

        for (int i = 0; i < r1; i++) {
            for (int j = 0; j < c2; j++) {
                C[i][j] = 0;
                for (int k = 0; k < c1; k++) {
                    C[i][j] += A[i][k] * B[k][j];
                }
            }
        }

        System.out.println("\nDisplay of the matrix");
        for (int i = 0; i < r1; i++) {
            for (int j = 0; j < c2; j++) {
                System.out.print(C[i][j] + "\t");
            }
            System.out.println();
        }
        scanner.close();
    }
}
```

Output:

```
D:\Java\Java P2 - Code>javac matrix.java

D:\Java\Java P2 - Code>java matrix
Enter rows for Matrix A: 5
Enter columns for Matrix A: 3
Enter rows for Matrix B: 5
Enter columns for Matrix B: 3

Matrix multiplication is NOT possible!
Columns of Matrix A must equal Rows of Matrix B.

D:\Java\Java P2 - Code>java matrix
Enter rows for Matrix A: 2
Enter columns for Matrix A: 2
Enter rows for Matrix B: 2
Enter columns for Matrix B: 2

Enter elements for Matrix A
A[0][0]: 1
A[0][1]: 2
A[1][0]: 3
A[1][1]: 4

Enter elements for Matrix B
B[0][0]: 5
B[0][1]: 6
B[1][0]: 7
B[1][1]: 8

Display of the matrix
19      22
43      50
```

B.2 Question of Curiosity:

1. Explain difference between while loop and do-while loop.

Feature	**while Loop**	**do-while Loop**
Condition Check	Checks the condition before executing the loop body (Entry-controlled).	Checks the condition after executing the loop body (Exit-controlled).
Minimum Executions	0 times if the condition is false initially, the code inside never runs.	At least 1 time the body always runs once before evaluating the condition.

Syntax	while (condition) { //code }	do { // code } while (condition);
Variable Access	Condition can't use variables declared inside the loop.	Condition can evaluate variables initialized/modified inside the do block.
Best Used For	Tasks where validation must happen before action (e.g., balance check before withdrawal).	Tasks where action must happen before validation (e.g., showing a menu at least once).

Use a while loop when you want to run code only if a condition is true from the start (e.g., checking if a user is logged in). Use a do-while loop when you need the code to execute at least once regardless of the condition (e.g., displaying a menu first, then asking for user input).

2. Explain if-else ladder with an example.

An if-else ladder is a control statement used to execute one block of code among multiple choices based on sequential conditions.

When a condition is tested:

- If it evaluates to true, its block of code runs, and the rest of the ladder is skipped.
- If it evaluates to false, execution moves down to test the next else if condition.
- If none of the conditions are true, the optional final else block executes.

The basic structure of what if-else ladder looks like:

```
if (condition1) {
   // Code runs if condition1 is true
} else if (condition2) {
   // Code runs if condition1 is false and condition2 is true
} else if (condition3) {
   // Code runs if condition1 and condition2 are false, but condition3 is true
} else {
```

```
    // Code runs if ALL above conditions are false
}
```

For example:

```
public class GradeChecker {
    public static void main(String[] args) {
        int marks = 82;
        if (marks >= 90) {
            System.out.println("Grade: A+");
        } else if (marks >= 80) {
            System.out.println("Grade: A");
        } else if (marks >= 70) {
            System.out.println("Grade: B");
        } else if (marks >= 50) {
            System.out.println("Grade: C");
        } else {
            System.out.println("Grade: Fail");
        }
    }
}
```

B.3 Conclusion:

Through the completion of these exercises, the main goal of learning how to manage control flow and structured data in Java was fully achieved. The practical work provided hands-on experience in using loops (for and while) to automate repetitive actions without rewriting code such as repeatedly displaying text, generating mathematical tables, calculating factorials, and summing up complex numerical series.

To handle collections of information, one-dimensional arrays were implemented to store multiple values under a single variable, making it simple to iterate through data to calculate totals and averages. Expanding on this, two-dimensional arrays were used to organize data into rows and columns, enabling structured record-keeping across multiple subjects to find top scores, as well as executing grid-based operations like matrix multiplication using nested loops.

On the theoretical side, a clear distinction was drawn between entry-controlled (while) and exit-controlled (do-while) loops, highlighting when to check conditions before running code versus guaranteeing at least one execution. Additionally, using if-else ladders showed how programs evaluate multiple conditions sequentially to make structured decisions. Overall, these exercises built a solid foundation in using core Java logic and arrays to solve practical programming problems efficiently.

****